

MediPilot Implementation README

Architecture, Tech Stack, and Execution

Team 01 BIT

August 30, 2026

Abstract

This document serves as the implementation README, outlining the solution architecture, technology stack, directory structure, synthetic data production, and the specific role of the Large Language Model (LLM). It incorporates the system’s architecture diagrams directly to provide a comprehensive, visual guide to the system.

Contents

1	Solution Architecture	2
1.1	Frontend (Web Layer)	2
1.2	Backend (Triage Engine)	2
2	Tech Stack	2
3	Role of the LLM	2
4	Production of Synthetic Data	2
5	Directory Structure	3

1 Architectural Implementation & Tech Stack

The MediPilot system enforces a strict boundary between perception and evaluation. The architecture is explicitly decoupled to guarantee that clinical risk engines and human interfaces operate in distinct, fail-closed isolation.

1.1 Track B: The Nurse & Patient Interfaces (Frontend)

The interface tier, located in `web/`, is a static Next.js 16 payload compiled to run as a pure client-side application. It comprises the seven clinical surfaces, including the interactive kiosk and the nurse board. This layer is entirely stripped of clinical authority. It manages the deterministic state machine of the branching triage conversation, raw speech-to-text integration, and local offline phrase matching. It cannot, by construction, evaluate risk.

1.2 Track C: The Triage Engine (Backend)

The backend, located in `backend/`, operates as a Python/FastAPI orchestrator. It receives structured records from the interface tier and evaluates them against a strict hierarchy of safety constraints. The orchestrator houses the deterministic Band Engine, the GBDT Risk Model, and the rigorous age stratification floors.

2 Technology Stack & Execution Constraints

The system executes on commodity hardware, complying with the strict data locality mandates of the DPDP Act 2023. It explicitly bans mandatory dependencies on paid cloud GPU infrastructure.

- **Frontend Interfaces:** Next.js 16, React 19, Tailwind CSS 4, utilizing native `webkitSpeechRecognition`.
- **Triage Orchestrator:** Python 3.10+, FastAPI, Uvicorn, Scikit-learn, Pandas.
- **Perception Models:** `faster-whisper` (CTranslate2, int8) and `Llama.cpp` for offline structural parsing.

3 The Confined Role of the Large Language Model (LLM)

The Large Language Model is strictly confined to **Track A (Perception Layer)**.

The LLM operates purely as a structural parser, translating unstructured patient narrative into a rigid JSON schema. It is structurally prohibited from producing an acuity score, assigning a triage band, or issuing clinical judgments. Should inference resources become constrained or the model output violate the schema, the system falls back to a deterministic, keyword-based structurer.

4 Synthetic Data Production

To validate the system's edge cases without exposing Patient Health Information (PHI), the demonstration corpus is entirely synthetic. The 20-record dataset (`web/lib/seed/corpus.ts`) is generated deterministically to test Wait-Ceiling breaches, Out-Of-Distribution (OOD) gates, and zero-history ABHA paths.

5 Repository Integrity & Structure

The repository is structured to enforce the separation of concerns between the architectural tracks:

```
.
|-- web/                # Track B: Next.js frontend (kiosk, board)
|-- backend/            # Track C: Python FastAPI orchestrator
|-- docs/               # Documentation and whitepaper specifications
|   |-- paper/          # LaTeX source files
|   |-- diagrams/       # SVG and PNG architecture diagrams
|   \-- submission/    # Compiled PDFs for evaluation
|-- archive/            # Quarantined legacy code
\-- .migration/         # Hashes proving migration integrity
```

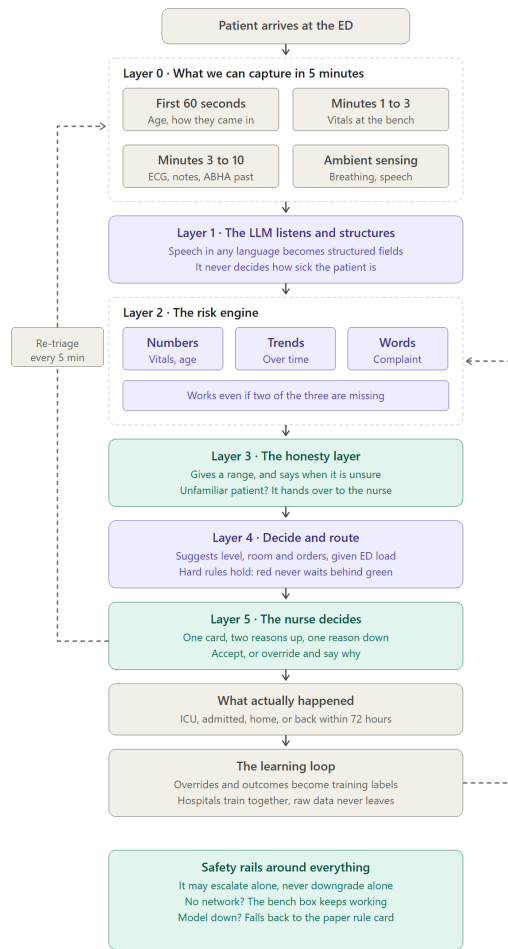


Figure 1: The Parallel Track Architecture: Perception (Track A), Interfaces (Track B), and Risk Engine (Track C).

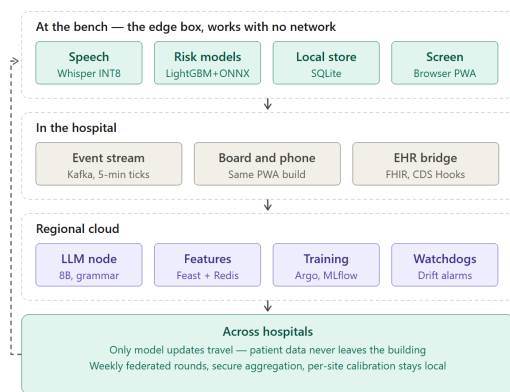


Figure 2: The Three-Tier Inference Hierarchy.

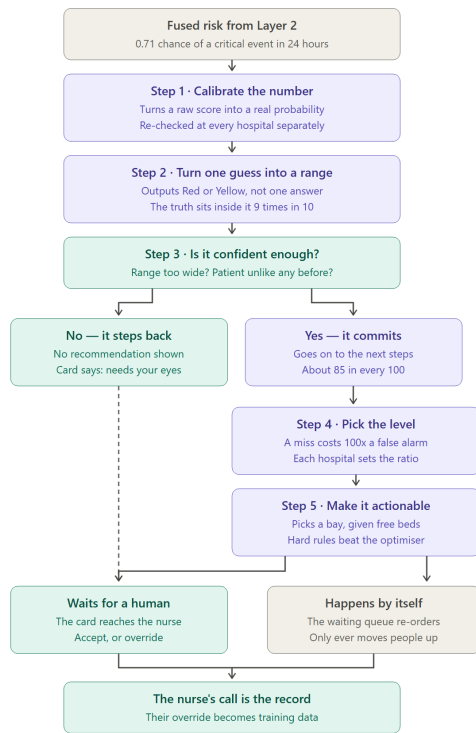


Figure 3: The Semantic Boundary: The model reports, the rule evaluates.